

Requerimientos en ambientes lean ágil

Introducción al Desarrollo ágil

Requerimientos en ambientes ágiles – User Stories

Lo más difícil del software es decidir que software queremos construir, ya que las cuestiones tecnológicas no son el impedimento, sino que los proyectos de software fracasan debido a que no se cumplen con las expectativas del cliente: lo que se construye no es lo que el usuario quería o necesitaba. Este problema se advierte en la crisis del software. Probablemente una de estas razones es no asimilar la complejidad que supone hacer software y que la etapa de relevar requerimientos (la cual es social y no técnica) suele ser subestimada.

También, es muy común que haya problemas de comunicación entre los clientes y el equipo. Cuando ninguno de los dos domina la comunicación, el proyecto fracasa. Cuando el cliente o negocio domina la comunicación, es probable que soliciten funcionalidades y fechas de entrega que los desarrolladores no puedan cumplir, o que no entiendan exactamente qué es lo que se requiere. Otro problema es cuando solo los desarrolladores dominan la comunicación, ya que la jerga técnica reemplaza el lenguaje del negocio y los desarrolladores pierden la oportunidad de aprender que es necesario escuchando al cliente.

Lo óptimo es que haya una comunicación de ambas partes, ya que los proyectos suelen fracasar si solo una parte domina la comunicación y asignación de recursos.

Como usuarios, ver “early versions” o entregas tempranas es una ventaja ya que permite que nos surjan nuevas ideas o que nos demos cuenta de que no es exactamente lo que queríamos antes de que el proyecto siga avanzando por un camino equivocado.

Es por esto que no es posible realizar un diagrama PERT perfecto mostrando todo lo que se debe realizar en un proyecto (no se conocen de arranque todas las actividades).

Fundamentación de los requerimientos ágiles

El fondo de la cuestión está en los principios y valores explicitados en el manifiesto ágil, fundamentalmente la entrega frecuente y temprana de software valioso y la posibilidad de recibir cambios de requerimientos aun en etapas finales. Esto último implica una predisposición, una actitud del equipo frente a la posibilidad de cambios. La base de los requerimientos en ambientes ágiles tiene que ver con la posibilidad de comenzar a construir el producto cuando el producto no está completamente definido.

Los requerimientos ágiles se basan en 3 pilares.

- **Usar el valor para construir el producto correcto:** Lo único importante es dejar contento al cliente con software funcionando que le sirva, por lo que, el foco de este enfoque de gestión ágil apunta a construir valor de negocio. Es decir, el software que creamos es para ayudar a alguien más a cumplir con sus objetivos y trabajo, por eso decimos que debemos enfocarnos en construir el producto correcto que le dé valor al negocio. Hay que construir un producto que le haga la vida más fácil a las personas que lo utilizan.
- **Determinar que es “sólo lo suficiente”:** hace referencia a que los requerimientos deben ser encontrados de a poco, no buscar todos los requerimientos desde un inicio y quedarnos solo con eso. Este pilar postula que se tiene que empezar con lo mínimo necesario para generar realimentación y alimentar el empirismo y a partir de allí, se avanza continuamente.
- **Usar historias y modelos para mostrar que construir:** la parte de desarrollo de requerimientos está muy enfocada a la idea de construir junto con el cliente, respetando el principio ágil de “técnicos y no técnicos trabajando juntos”. Se busca trabajar con un referente que esté del lado del negocio, que tenga el conocimiento del producto y que trabaje junto con el equipo.

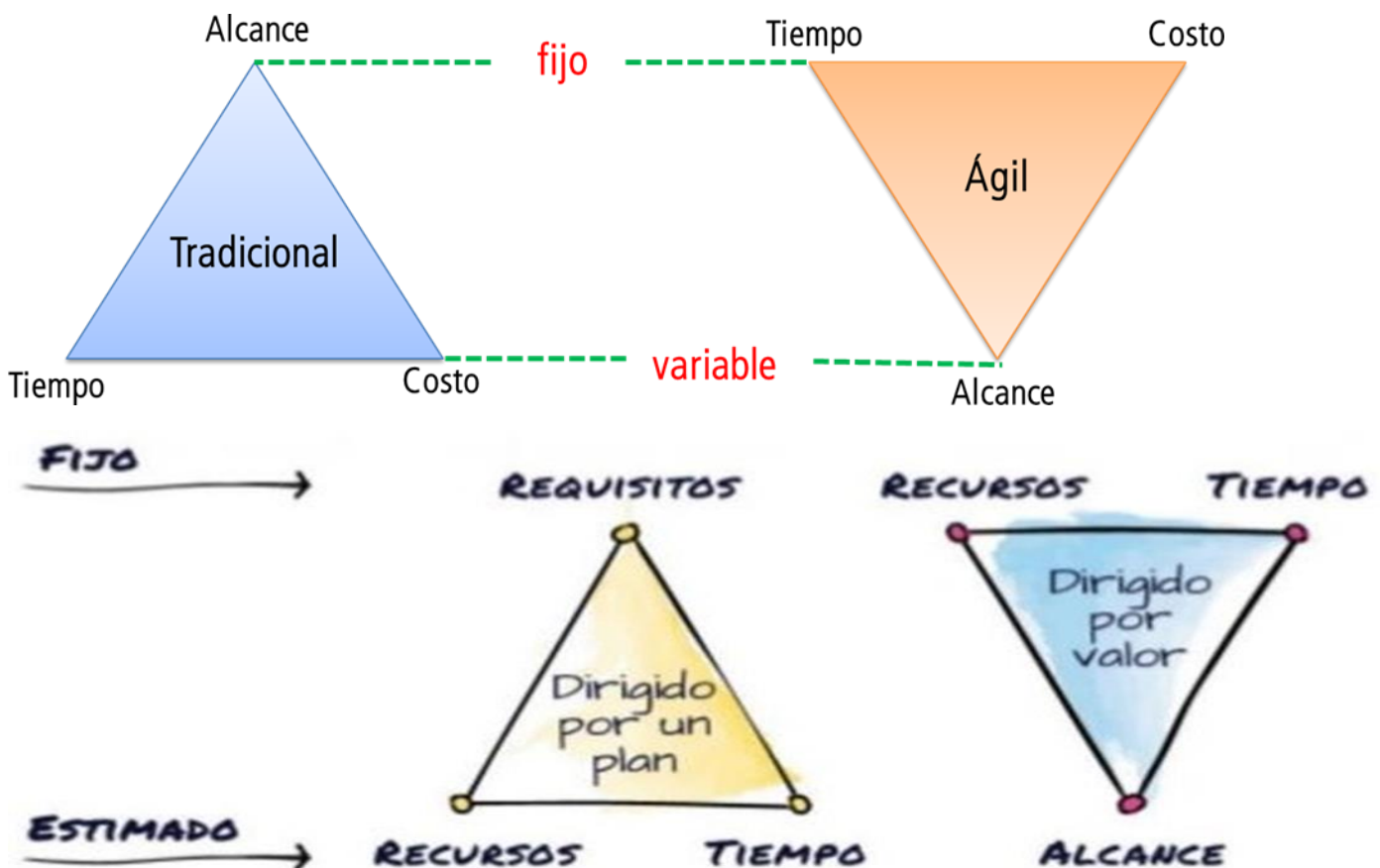
La etapa de capturar los requerimientos se transforma en una etapa de trabajo muy integrado entre el cliente y nosotros. Por eso el mejor medio de comunicación es la comunicación cara a cara porque la idea es armar equipos de trabajo en los que con técnicas de descubrimiento (las User Stories) se pueda lograr crear la visión de qué producto se quiere construir y a partir de ahí, avanzar.

El enfoque de requerimientos ágiles debe estar adherido al manifiesto ágil, es decir, a cumplir con los valores y con los principios que el manifiesto plantea.

Recordemos que la ingeniería de requerimientos tiene 2 procesos: por un lado, **el desarrollo de requerimientos** y por otro lado **la gestión de los requerimientos**. La gestión está más vinculada con mantener los requerimientos íntegros, bien identificados, bien trazados y que si cambian los requerimientos se actualicen todos los lugares donde se tengan que actualizar esa definición de requerimientos. Esto ocurre todo el tiempo mientras estamos trabajando con un producto de software. La parte de desarrollo de requerimientos, que es la parte que tiene que ver con identificar, especificar y validar los requerimientos.

La triple restricción – enfoque ágil

En el enfoque ágil, se trata de aceptar la realidad en la cual el alcance (requerimientos) del producto puede cambiar a lo largo del desarrollo, y no es posible definirlos al comienzo de este. Entonces, el enfoque trata de dejar el tiempo y los recursos fijos, teniendo en cuenta “cuánto software” le puedo entregar en tanto tiempo y con tal equipo de desarrollo.

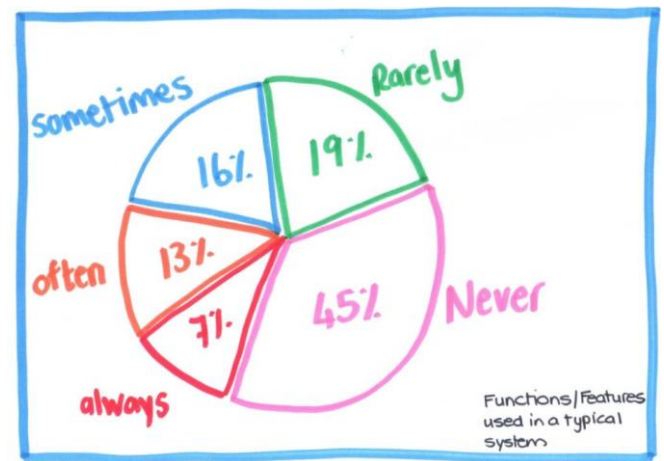


Por otro lado, en el enfoque tradicional, el alcance se fija y en función de eso derivan los recursos (costo) y el tiempo. El cliente desea controlar el alcance, ya que este comprende lo que él realmente necesita. Luego el cliente pregunta por el tiempo y el costo, los cuales son derivados por el equipo. Por esta razón es tan costosa incorporar cambios de requerimientos, ya que se estaría modificando el alcance y como consecuencia los recursos y el tiempo.

BRUF – Big Requirements Up Front

La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software. A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.

Además, es prácticamente imposible conocer al 100% todas las necesidades de los usuarios, ya que ni ellos saben concretamente qué necesitan, ni cómo explicar de forma abarcadora y completa sus necesidades. Por otro lado, esas necesidades que tienen los usuarios al momento de ser relevadas, con el tiempo probablemente irán cambiando, con lo cual, la priorización es un factor fundamental en esa etapa, donde el equipo en conjunto con el Product Owner, deberán ser capaces de decidir cuáles son aquellos requerimientos indispensables para el negocio, de manera que, si se implementan esas necesidades, se puede entregar una nueva versión del producto que sea de valor para el cliente.



Product Backlog

La gestión ágil busca contrarrestar la situación planteada con anterioridad donde gran parte del software no se termina utilizando. El Product Backlog es una lista priorizada de características y requerimientos del software. Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales. El Product Owner se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.

En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software, se terminaba construyendo un producto que no dejaba satisfecho al cliente. En ágil es el cliente el encargado de esta priorización, lo que lleva a que se construya el producto deseado.

Con esta situación se permite la compensación de no tener documentos de requerimientos formales escritos, detallando todos los puntos. Ágil ofrece una compensación al afirmar que la mejor comunicación es el cara a cara y un cliente participe en el equipo.

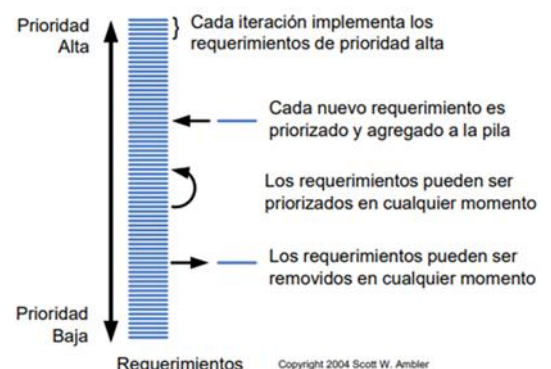
Requerimientos Just in Time

La gestión ágil de requerimientos sostiene los **requerimientos justo a tiempo**, asociados a los principios de eliminar el desperdicio y diferir compromisos, dentro de la filosofía Lean.

Se empieza con una visión del producto y con algunos requerimientos identificados. A partir de esta base, se comienza a construir el producto y se incorpora al cliente para obtener retroalimentación. **El Product Backlog nunca está al 100%**

En el enfoque tradicional se aplica el "Up Front Specification", mediante el cual se especifica todos los requerimientos antes de comenzar el desarrollo. Esto implica que un error en la captura de requerimientos tiene un alto impacto. En cambio, los requerimientos en ambientes ágiles al ser consensuado continuamente con el cliente, el impacto que se tiene es menor.

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



El producto no va a estar especificado al 100% desde el principio ni muchísimo menos, sino que se van a ir encontrando requerimientos y describiendo los requerimientos que hacen falta conforme haga falta. “*ni antes, ni después*”.

Fuente de requerimientos

Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos del software se obtienen conversando cara a cara con el cliente.

En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo (sería lo que en el gráfico dice “papel”).

Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Esto permite reducir la cantidad de documentación formal. La elicitación de requerimientos es un proceso que requiere de conversar de forma personal, analizando lo que los clientes dicen y los gestos que manifiestan.

El desarrollo ágil no implica la no documentación o la no especificación de requerimientos, sino que prioriza documentar y especificar aquellos que sean de valor para el cliente, antes que hacer toda una ERS errónea.



Momento de la captura de requerimientos

En el enfoque tradicional, los requerimientos son definidos al principio del proyecto. El caso particular del PUD, que es un proceso definido con un modelo de proceso iterativo e incremental, el flujo de trabajo de requerimientos tiene un gran peso en la fase inicial y de elaboración.

En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto. El inicio de la construcción del software se da en un contexto en el cual los requerimientos no están completamente definidos, sin embargo, se tiene una visión.

Contenedor y persistencia de los requerimientos

El enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS. En definitiva, es el documento de más alto nivel que define las características del sistema.

En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de los requerimientos, ya que una vez implementados estos se “persisten” en el producto funcionando. El detalle de valor queda persistente en los casos de prueba que se construyen en el Testing.

Tipos de requerimientos

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

A nivel de la gestión ágil de requerimientos se trabaja con:

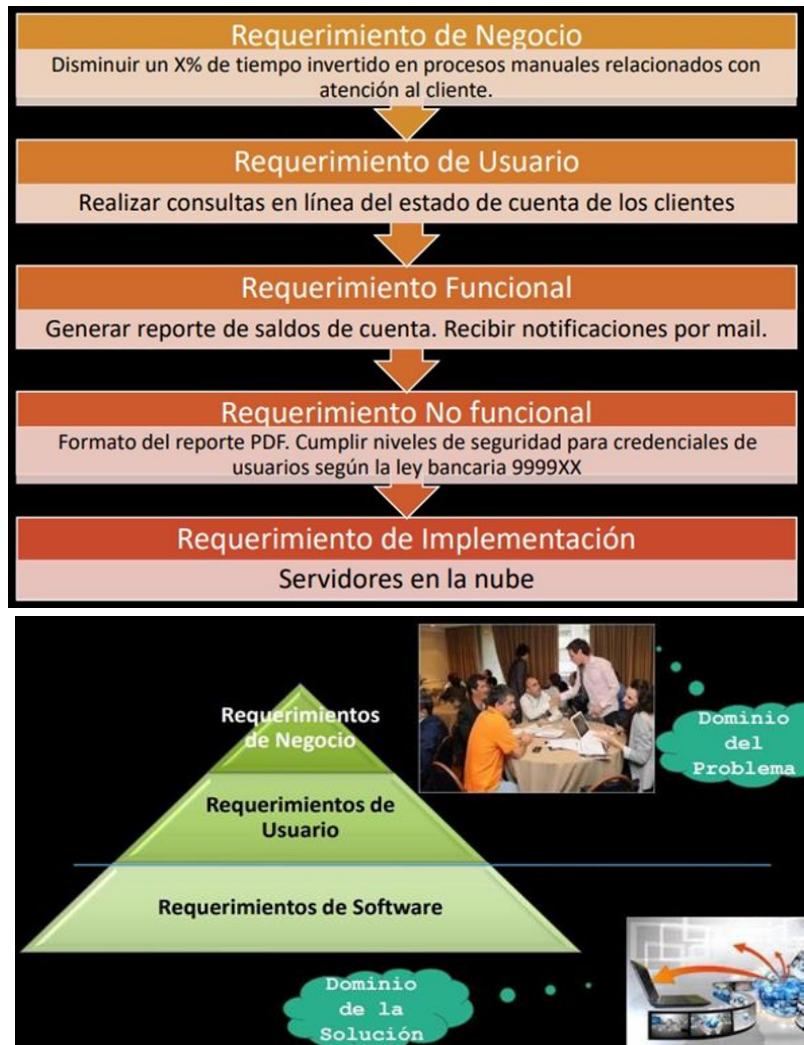
- Requerimiento de negocio
- Requerimiento de usuario

Las User Stories son requerimientos de usuario alineados a un requerimiento de negocio.

Al usuario le interesa el requerimiento de negocio. El valor de negocio es el requerimiento de negocio, pero el requerimiento de usuario es el medio por el cual vamos a lograr ese valor de negocio.

Las User Stories no sirven para especificar requerimientos de software, sirven para identificar requerimientos de usuario.

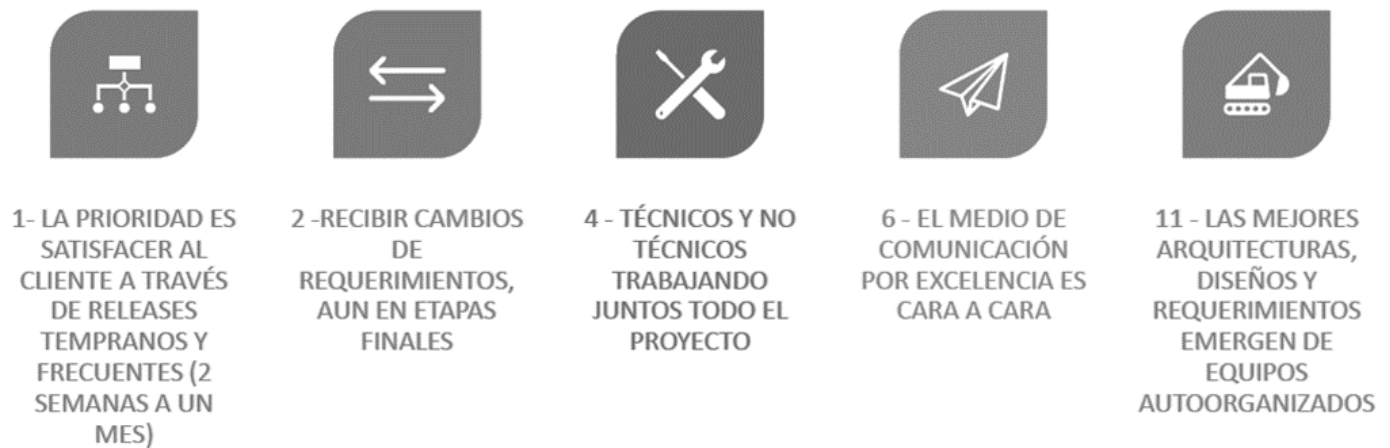
El foco de la gestión ágil de requerimientos está puesto sobre el usuario y sobre sus necesidades y por eso en enfoques como scrum quién debe trabajar para identificar las User Stories es el Product Owner. Ya que es él quien se concentra en las necesidades de usuarios que van a satisfacer las necesidades de negocio.



¿Qué debemos asumir?

- Los cambios son la única constante. Todo el tiempo va a haber cambios en los requerimientos.
- Stakeholders: no son todos los que están. Debemos saber quiénes son los involucrados y los involucrados son todos los que tiene algo que decir respecto del producto. El PO es el representante de todas estas personas y el que se comunica con todas ellas para luego ser él mismo el que se comunica con el equipo que hace el software.
- Siempre se cumple eso de que: “El usuario dice lo que quiere cuando recibe lo que pidió”.
- No hay técnicas ni herramientas que sirvan para todos los casos. Debemos detectar para cada situación cuál es la herramienta más adecuada.
- Lo importante no es entregar una salida, un requerimiento, lo importante es entregar, un resultado, una solución de “valor”.

Principios ágiles relacionados con requerimientos ágiles



User Stories

Son una técnica para trabajar requerimientos de usuario en ambientes ágiles. La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.

Funciona como un recordatorio para el equipo de desarrollo y el Product Owner de que tienen que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor. En otras palabras, es un mecanismo para diferir una conversación.

Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario. Las US son escritas por el Product Owner.

Internamente, las User Stories se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.

Se dice que son multipropósito, debido a que cumplen diferentes funciones en el desarrollo de un software:

- Representan una necesidad de usuario, de forma no detallada.
- Conforman una descripción del producto.
- Son utilizadas como ítems de planificación.
- Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
- Junto con el código ejecutable, permiten documentar el producto.

La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas. Cada una tiene un interés respecto a esta información.

La definición de los requerimientos es lo que más afecta la calidad del sistema final y es lo más complejo de definir con detalle debido a la incertidumbre y cambio constante que sufren, que se da por el problema de comunicación. La dificultad se basa en qué se escribe y cuándo.

La elicitación tradicional no da buenos resultados, ya que todas las decisiones se toman al principio del proyecto cuando menos información se tiene. Por lo que se estableció que esta toma de decisiones se traslada a lo largo del proyecto, a medida que la incertidumbre baja y la información sube, buscando que se obtenga más a menor tiempo posible. Aquí surgen las User Stories.

Partes de la User Story

Está compuesta por tres partes:

1. **Tarjeta:** es una descripción de la historia, utilizada para planificar y como recordatorio. Tiene una sintaxis particular que es la siguiente:

<<Frase Verbal>>
Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
 - Actividad: representa la acción que realizará el sistema.
 - Valor de negocio: es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el Product Owner escriba las US. Permite priorizar el desarrollo.
 - Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.
 - Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad.
Se deben redactar independientes a la implementación (alto nivel de abstracción).
 - Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.
2. **Conversación:** Es la parte más importante de la US, ya que explicita la comunicación entre el Product Owner y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US.
El Product Owner es parte del equipo, por lo cual se espera responsabilidad, compromiso y confianza de su parte para hacerse cargo y llegar a un acuerdo común con el equipo sobre lo definido acerca de la funcionalidad.
También es posible dejar la conversación persistente en grabaciones, minutas o softwares de gestión que posibiliten la conversación. De todas maneras, la conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación.
En simples palabras, son discusiones acerca de la historia que sirven para desarrollar los detalles de la historia.
 3. **Confirmación:** Definición de un acuerdo entre el Product Owner y el Equipo para demostrar que la característica del producto realmente se implementó. Conformar a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Ventajas

- Prioriza la comunicación verbal por sobre la escrita (recordar la conversación es la parte más importante).
- Son comprensibles tanto para el cliente/usuario (Product Owner) como para el equipo de desarrollo (recordar que se escriben en términos de negocio).
- Tienen el tamaño justo para permitir planificar.

- Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
- Permiten desestimar los detalles, hasta que se logre una mejor comprensión sobre cuál es la necesidad concreta de los usuarios.

Desventajas

- En proyectos grandes, es difícil entender la relación entre U.S (son independientes entre sí).
- No son apropiadas para equipos demasiado grandes (por el tema de priorizar la conversación verbal, por sobre la escrita).

Product Backlog y las User Stories

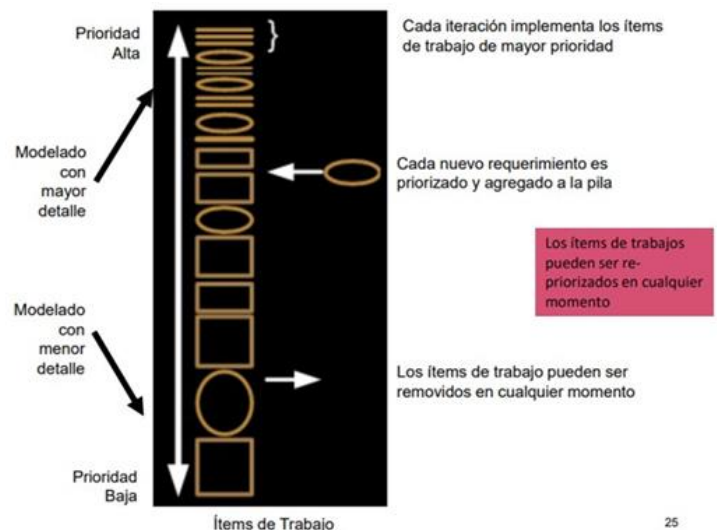
Es una pila de PBIs (Product Backlog Ítems) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (que son aquellas U.S. con granularidad fina), y las de menor prioridad (normalmente aquellas User de granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el Product Owner, quien es el encargado de realizar el proceso de priorización.

En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.

Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila (las de mayor prioridad que están arriba) se quitarán para implementarlas en la próxima iteración.

Debido a estas dos situaciones (agregar y quitar ítems), nunca encontraremos el 100% de los requerimientos de usuario en el Product Backlog.

El Product Owner
Prioriza las
historias
en el
Product
Backlog



25

Porciones verticales

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos.

Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.

Story 1	Story 2
GUI	
Business Logic	
Database	

Tamaño

Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.

El tamaño de las U.S. se mide en Story Points.

Hay 3 aspectos que determinan el tamaño de una U.S.:

- Complejidad: hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas. Por ejemplo, algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.
- Esfuerzo: es el tiempo en horas de trabajo que requerirá la implementación.
- Incertidumbre
 - Técnica: cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
 - De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Points

Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales miden el tamaño de un producto, NO de una medida basada en el tiempo (por ejemplo, horas hombre). Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores.

Lo relativo surge a partir de que se comprobó que las personas no saben estimar en términos absolutos, sino que les resulta útil la comparación de tamaños.

Niveles de abstracción

1. **User Story:** cumple con el criterio de Listo (DoR) para ingresar a una iteración.
2. **Épica:** es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User.
Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST. La épica debe ser dividida en US más pequeñas.

Sirven para aquellos requerimientos que no son prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas.

Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.

3. Temas: son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Ambos (Épicas y Temas) permiten reducir los esfuerzos del equipo a la hora de estimar.



Modelado de Roles

Se utilizan las **tarjetas de rol de usuario**, aquí se anotan las características en general que va a tener el usuario.

El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.

Técnicas adicionales

1. **Personas:** esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
2. **Personajes extremos:** que personas en términos extremos podrían llegar a utilizar el software.
3. **Proxies (Usuarios Representantes):** es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de aceptación

Es una nota no obligatoria que define un acuerdo respecto a cómo se debe comportar el software para que el PO acepte la implementación, es recomendable que este debido a que sino los desarrolladores no tendrían un parámetro para definir si la misma fue cumplimentada o no. Lo fundamental es que esté escrito en términos objetivos y concretos.

El criterio de aceptación es información concreta que tiene que servirnos a nosotros para saber si lo que implementamos es correcto o no. Debemos saber si el PO nos va a aceptar esta característica implementada o no, porque la va a aceptar si respeta los criterios de aceptación.

- Definen límites para una User Story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos).
- Ayudan a que el equipo tenga una visión compartida de la US.
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuándo parar de agregar funcionalidad en una US.

¿Cuáles son los criterios de aceptación buenos?

- Definen una intención, no una solución. Ejemplo: El usuario debe elegir al menos una cuenta para operar.
- Son independientes de la implementación.
- Relativamente de alto nivel, no es necesario que se escriba cada detalle.

¿Y los detalles, a dónde van?

Detalles como:

- El encabezado de la columna se nombra "Saldo"
- El formato del saldo es 999.999.999,99
- Debería usarse una lista desplegable en lugar de un Checkbox.

Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:

- Documentación interna de los equipos

- Pruebas de aceptación automatizadas.

Cada equipo decide dónde guardar el detalle y cómo lo va a mantener.

Pruebas de aceptación

Son declaraciones de intención de que hay que probar. El conocimiento del negocio está inmerso en las pruebas de aceptación. Se prueban escenarios exitosos y escenarios que fallen. En las pruebas de aceptación se encuentran las más importantes, ya que **no es posible escribir todas las posibles pruebas de aceptación**.

El Product Owner acepta la User como implementada si todas las pruebas de aceptación se cumplen.

Expresan detalles resultantes de las conversaciones entre clientes y desarrolladores; suelen usarse para completar detalles de la US. Es importante mencionar que las mismas deben escribirse antes que la programación empiece, y que se recomienda que las escriba el cliente.

DoR (Definition of Ready) – Definición de Listo

Es una medida de calidad que **determina si la User Story está en condiciones de entrar a una iteración de desarrollo**. Para que la US pueda ser implementada, **mínimamente debe satisfacer el INVEST Model**. Entre el equipo de desarrollo y el PO pueden definir características o condiciones extras al INVEST para que una US se considere lista para entrar a la iteración de desarrollo.

INVEST Model

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

- **Independiente:** la User es calendarizable e implementable en cualquier orden. Esto implica que no existe una dependencia con otra User para desarrollarla o para mostrarla al Product Owner. El PO tiene la libertad de priorizarlas en el Product Backlog como él desee.
- **Negociable:** Las US no son contratos estrictos, sino más bien pequeñas descripciones de funcionalidades donde se describen las necesidades de los usuarios, y los detalles se dejan para negociar en conversaciones futuras entre clientes y el equipo de desarrollo. Esto implica que no deben detallarse todos los aspectos de forma exhaustiva, sino más bien, servir a modo de recordatorio para negociar aquellos aspectos en un futuro cuando se deba implementar.
El cliente debe expresar lo que necesita, es decir que está centrada en el “qué” no el “cómo”. El cómo es decisión del equipo y debe ser negociable con el Product Owner.
La US debe estar escrita en términos de qué necesita el usuario y no de cómo lo vamos a implementar.
- **Valuable:** Una US debe ser valorable para los clientes o usuarios (el para qué de la descripción). Es decir, las U.S. deben aportar valor de negocio a quiénes estará destinado el producto, no para el equipo de desarrollo. Esto implica dejar de lado en US, por ejemplo, las tecnologías utilizadas para el desarrollo del producto.
- **Estimable:** la US debe contener la cantidad de información suficiente para estimar el tamaño, la complejidad y el esfuerzo necesario para realizarlo. Sin poder estimarla no es posible determinar si se puede finalizar en una iteración por lo tanto no puede ser construida (No puedo saber cuánto tardaría).
Hay tres razones por las cuales una US puede no ser estimable:
 1. El equipo tiene incertidumbre acerca del dominio.
 - a. Deberán discutir con el Product Owner para esclarecer todas las dudas.
 2. El equipo tiene incertidumbre técnica, es decir, sobre el dominio de la solución.
 - a. Deberá destinarse uno o más miembros del equipo a uno o más Spikes.
 3. La US es demasiado grande.

a. En este caso, deberá dividirse en US más pequeñas.

- **Small (Pequeña):** la US debe ser lo suficientemente pequeña como para ser finalizada en una iteración. Se debe comenzar y terminar una característica en una iteración, no debe haber trabajos a medias. El tamaño es relativo. ya que depende del equipo, de la experiencia que tiene, de su capacidad de trabajo y también depende de la duración de la iteración. Ágil aplica gestión binaria, es decir, si se puede completar la US en la iteración, es pequeña. No aplica porcentaje de avances.
- **Testeable:** la US debe cumplir con las pruebas de aceptación para poder probar si la misma fue o no implementada (se muestra que está hecha y demostrable).

DoD (Definition of Done) – Definición de Hecho

Determina si la US está terminada. Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no. En el momento que una U.S. cumple con todo lo pactado en esta Definición de Hecho, se considera que esa US está lista para ser desplegada y aportar valor al cliente.

Esto permite crear transparencia, debido a que proporciona a todo el equipo una comprensión exhaustiva de qué trabajo se realizó, para lograr ese incremento.

En la práctica consta de un Checklist de condiciones que deben cumplirse. Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una Definición de Hecho adecuada para el producto.

Spikes

Son un tipo especial de US que **se producen por la incertidumbre** que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo, Una vez resuelta la incertidumbre, la Spike se convierte en una o más US. Es una característica más del producto a la cual el equipo le tiene que dedicar tiempo para:

1. Familiarizarse con una nueva tecnología o dominio.
2. Investigar y prototipar para ganar confianza frente a:
 - a. Riesgos tecnológicos.
 - b. Riesgos funcionales, donde no está claro cómo debe reaccionar el sistema para satisfacer las necesidades del usuario.

Se usan para quitar el riesgo e incertidumbre de una US y otra faceta del proyecto. Se clasifican en técnicas y funcionales y pueden utilizarse para:

1. Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
2. Analizar el comportamiento de una historia compleja y poder dividirla en piezas manejables.
3. Ganar confianza frente a riesgos tecnológicos, investigando o prototipando.
4. Ganar confianza frente a riesgos funcionales, donde no está claro cómo el sistema debe resolver la interacción con el usuario para alcanzar el beneficio esperado.

Spikes técnicos

Utilizadas para evaluar diferentes enfoques técnicos y tecnológicos en el dominio de la solución. Está asociado a la implementación de la funcionalidad. Nueva tecnología.

Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo. Dependen de los técnicos (nosotros).

Spikes funcionales

Utilizada cuando hay cierta incertidumbre respecto de cómo el usuario interactúa con el sistema, usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios involucrados.

Dependen de definiciones del negocio, del PO.

Lineamientos para Spikes

- Estimables, demostrables, y aceptables.
- La excepción, no la regla.
 - Toda historia tiene incertidumbre y riesgos.
 - El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
 - Los spikes deben dejarse para incógnitas más críticas y grandes.
 - Utilizar spikes como última opción.
- Implementar la spike en una iteración separada de las historias resultantes.
 - Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.



DIFERIR EL ANÁLISIS
DETALLADO TAN TARDE
COMO SEA POSIBLE, LO QUE
ES JUSTO ANTES DE QUE EL
TRABAJO COMIENCE.



HASTA ENTONCES, SE
CAPTURAN
REQUERIMIENTOS EN LA
FORMA DE "USER STORIES".



LAS USER STORIES NO SON
REQUERIMIENTOS DE
SOFTWARE, NO NECESITAN
SER DESCRIPCIONES
EXHAUSTIVAS DE LA
FUNCIONALIDAD DEL
SISTEMA.

Investment Themes (Temas de Inversión)

Representan un conjunto de iniciativas o propuestas de valor que conducen la inversión de la empresa en sistemas, productos, aplicaciones y servicios con el objetivo de lograr una diferenciación en el mercado y/o ventajas competitivas.

Los THEMES son una combinación de inversión en:

1. Inversión en ofertas de productos existentes, mejoras, soporte y mantenimiento.

2. Inversión en nuevos productos y servicios que mejorarán los beneficios y/o ganarán porciones de mercado en el período actual o al corto plazo.
3. Inversión a futuro en productos y servicios avanzados.
4. Inversión hoy, pero que no dará beneficios hasta dentro de unos años.
5. Inversión en reducción (estrategia de retiro) para ofertas existentes que están cerca del final de su vida útil.

Estimación en ambientes ágiles

Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto. Se relaciona con los objetivos del negocio, compromisos y control.

Según McConell una estimación en el contexto del software es una predicción de cuánto tiempo durará o costará un proyecto. Es considerada una de las actividades más complejas en el desarrollo de software.

Las estimaciones no son planes ni compromisos, sino que sirven de base para planificar, pero el plan final no tiene por qué coincidir con lo estimado.

Errores en las estimaciones

1. Información imprecisa acerca del software a estimar o acerca de la capacidad para realizar el proyecto.
2. Demasiado caos en el proyecto (mal definido el proyecto).
3. Imprecisión generada por el proceso de estimación.
4. Una de las fuentes más común es omitir actividades necesarias para la estimación del proyecto tales como, requerimientos faltantes, licencias, reuniones, revisiones, etc.
5. Se olvidan del detalle que planear no es estimar y estimar no es planear, sino que las estimaciones son la base de los planes, pero los planes no tienen que ser lo mismo que lo estimado.
6. A mayor diferencia entre lo planeado y lo estimado, mayor el riesgo.
7. Las estimaciones no son compromisos.

¿Por qué estimamos?

Existen dos razones principales por las cuales se llevan adelante las estimaciones:

- Para predecir completitud.
- Para administrar riesgos.

Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el **trabajo** que habrá de realizarse, los **recursos** que se requerirán y el **tiempo** que transcurrirá desde el principio hasta el final. Las estimaciones requieren:

- Experiencia.
- Acceso a buena información histórica (métricas).
- El valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe.

Siempre se desea saber muy al inicio del proyecto cuál será el costo, el tiempo que llevará, entre otros aspectos y como aún no se conoce con precisión cuál será el alcance del producto resulta complejo predecir con exactitud estos aspectos, es por ello que a medida que avanza el proyecto, las estimaciones son más certeras, dado que se cuenta con

mayor información sobre la cual realizar las estimaciones. (Es por ello que se suele decir que típicamente la primera estimación difiere hasta un 400%).

Al inicio del desarrollo hay una estimación errónea de 2 a 4 veces más, esto se debe a dos aspectos:

1. Al inicio del proyecto tenemos poca información y demasiada incertidumbre, lo que aumenta el riesgo de generar estimaciones erróneas.
2. Las estimaciones son de naturaleza optimista, por eso las estimaciones deben ser recalculadas conforme se avanza con el proyecto.
3. Se suele decir: “el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces”.

Proceso de estimación

Para ver cómo y qué se estima en el ambiente tradicional ver el punto de estimación en el título “[Plan de proyecto](#)” en la unidad 1 de este apunte.

Métodos utilizados para estimar

Se recomienda utilizar diferentes métodos de estimación y luego contrastar (comparar). Todos tienen un “Factor de Ajuste” que permite que el método cierre (calibraciones).

1. Basados en la experiencia

- a. **Datos históricos:** consiste en recolectar datos básicos de otros proyectos para ir generando una base de conocimientos que sean de utilidad para futuras estimaciones, con esto se busca una alternativa en la cual el conocimiento de cada individuo se transfiere a la organización y no estamos atados a una única persona. Se deben utilizar para no tener problemas entre desarrolladores y clientes.
Esto no va acorde a la gestión ágil, que dice que la experiencia no se puede extrapolar a otros equipos.
 - i. **Industry Data:** datos de otras organizaciones que aportan al mercado y desarrollan productos con algún grado de semejanza y que permite una comparación básica.
 - ii. **Historical Data:** datos de la organización de proyectos que se desarrollaron y ya se cerraron.
 - iii. **Project Data:** datos del proyecto, pero de etapas anteriores a la que se está estimando.
- b. **Juicio experto:** es uno de los más utilizados, el problema se presenta cuando se utiliza como única técnica de estimación el juicio experto puro. Existen dos técnicas para aplicarlo:
 - i. **Juicio experto puro:** un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (el problema es que, si el experto se va de la organización, esta pierde su capacidad de estimación).
Se puede utilizar el “método de optimista, pesimista, habitual”, en el que se le pregunta al experto sobre 3 valores distintos y se aplica una fórmula para calcular la estimación.
 - ii. **Juicio Wideband Delphi:** es similar al anterior pero grupal, un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración. Hay un coordinador que va a reunir las estimaciones individuales de cada uno de los expertos cuando les dieron las especificaciones, se opina sobre las estimaciones ajenas de manera anónima, se vuelve a discutir, revisan, las vuelven a enviar al coordinador y se repite hasta que todos estén de acuerdo de forma razonable. Este método es tomado como base para el Poker Planning.

2. Analogía

Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares. Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro. Es decir, consiste en tomar a un proyecto e ir comparando factores, por ejemplo:

- Tamaño: es menor o mayor al proyecto anterior.

- Complejidad: más complejo de lo usual.
- Usuarios: si hay más usuarios habrá más complicaciones, como la concurrencia o los datos a almacenar.

3. Otros

- Basados exclusivamente en los recursos:** consiste en analizar el personal que se tiene y de cuánto tiempo se dispone del mismo, haciendo las estimaciones exclusivamente en función de este factor. En la realización: “El trabajo se expande hasta consumir todos los recursos disponibles, independientemente si se alcanzan o no los objetivos.”
- Basados en la capacidad del cliente o exclusivamente en el mercado:** ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago. En este método no se evalúa al producto que se está intentando vender, sino a quien se lo vende. A veces se cobra menos para que nos elijan antes que a la competencia.
- Basados en los componentes del producto o en el proceso de desarrollo:** se pueden aplicar 2 enfoques:
 - Bottom-up:** se descompone el producto en unidades lo más pequeñas posible. Se estima cada unidad y luego se hace la estimación total.
 - Top-down:** se descompone el producto en grandes bloques y luego se estima cada uno de los componentes.
- Métodos algorítmicos:** se ejecutan algoritmos que tienen como parámetros de entrada medidas cuantitativas de tamaño, complejidad y conocimiento de dominio. Cada variable tendrá asociada un factor multiplicativo y en función de esto se obtiene como resultado un valor de esfuerzo. Es importante destacar que, a pesar de ser un cálculo matemático, puede no ser certero igual.

Problemas de estimaciones tradicionales

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

Estimaciones en ambientes ágiles

Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra. La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el Product Owner, no participa en estimaciones). Al ser de esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros, haciendo que ese proceso de estimar se vuelva lo más preciso posible (a diferencia de las metodologías tradicionales, donde se estima individualmente y se depende sólo de ese miembro).

Para estimar de esta forma, es fundamental que los miembros del equipo tengan una actitud en escucha, donde todos puedan opinar libremente y no haya influencias que “tapen” la opinión de otros.

Frente a la posibilidad de estimar mal y generar enojo por parte del cliente, tenemos dos reacciones posibles:

Como la estimación es un trabajo a riesgo, se subestima, o sea, simplificando sin considerar todo el trabajo que hay que hacer, por miedo a que si el valor es más alto el cliente lo rechace.

Sobreestimar, cuando me preguntan por ejemplo cuantas horas me va a llevar un trabajo, y, sabiendo que me va a llevar 2 horas, digo 4 por las dudas pase algo en el medio, inflando muchísimo la complejidad del producto.

Las estimaciones se tienden a confundir con compromisos o planificación. Uno planifica basándose en la estimación, pero no son lo mismo.

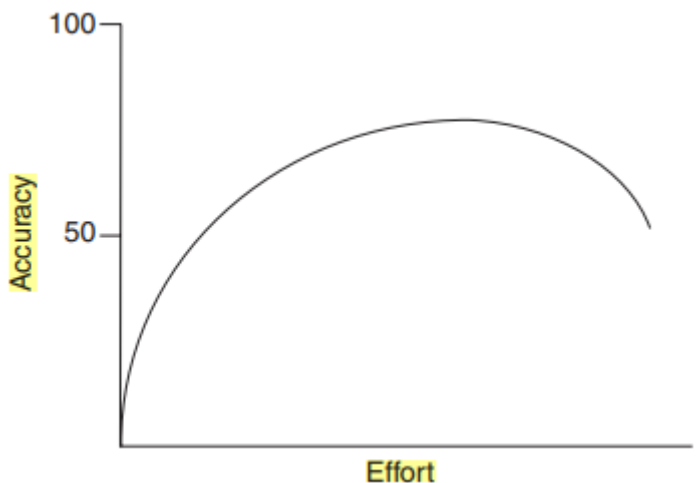
No es una instancia única, ya que a medida que se avance el proyecto, más información vamos a tener y más eficiente van a ser nuestras estimaciones.

Teniendo en cuenta esto, hay que aceptar que la estimación no es fácil, de hecho, es lo más difícil luego de la elicitación.

El gran beneficio del proceso de estimación es justamente “hacerlas”. Al estimar, se enriquece el trabajo del equipo, lo cual mejorará futuros procesos de estimaciones. Además, este proceso ayuda a determinar la factibilidad del trabajo planificado en etapas tempranas.

En el enfoque ágil, el esfuerzo destinado para estimar es bajo, pero amparándose en el concepto de que, al destinar poco esfuerzo en la estimación, se logra acercarse bastante a la realidad, la cual no sigue una relación lineal: si destinamos demasiado esfuerzo a este proceso, no obtendremos una precisión casi perfecta. Es por esto, que, en este enfoque, el tiempo de estimación se define y asigna con anterioridad para no utilizar más esfuerzo del necesario.

Hay que recordar que el remedio, debe ser más barato que la enfermedad: el objetivo es estar en alguna parte de la izquierda del gráfico, donde con poco esfuerzo, se alcanzan grandes valores de certeza en la estimación. “La precisión es cara”.



Existe una serie de consideraciones importantes:

- Son medidas relativas no absolutas.
- No es una medida basada en el tiempo.
- Las personas no saben estimar en términos absolutos, somos buenos comparando.
- Comparar es generalmente más rápido.
- Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
- Se emplea mejor el tiempo de análisis de las Stories.

Hay una fuerte propuesta de que estime la misma persona que va a realizar el trabajo. Se debe tener la mente abierta para escuchar lo que el resto del grupo tiene para decir y no cerrarse a nada. Las estimaciones son un proceso y uno tiene que hacer foco en este proceso.

Lo ideal es comenzar con una estimación que se pueda revisar todas las veces que sea necesario. Recordando el principio de que “la mejor métrica de progreso es el software funcionando”, debemos entender que todo lo definido en cuanto a las estimaciones tiene que ver con el producto.

No se puede comparar una User Story con otra porque hay distintas dimensiones y ahí es donde entra el Story Point, que hace de unidad homogeneizadora que permite comparar.

Para las estimaciones se deben tener en cuenta tres elementos a la hora de estimar: complejidad, esfuerzo e incertidumbre.

Estas tres variables son las que conforman al Story Point y de esta manera podemos comparar las US.

¿Qué se estima?

Lo que se estima en ambientes ágiles, es solo el tamaño de las US (o Features) usando como unidad de medida los Story Points, y teniendo en cuenta 3 aspectos de estas: incertidumbre, complejidad y esfuerzo. Como la complejidad de las US tiende a incrementarse exponencialmente, se utilizan comúnmente series numéricas de esta índole, siendo una de las más comunes, la serie de Fibonacci.

Al ser estimaciones de tamaños relativos, (es decir, se estima realizando comparaciones de los tamaños de las US) lo importante en este proceso no es el valor que se asigna a cada US en particular, sino más bien que las estimaciones sean consistentes en el conjunto. Es decir, para un equipo, una US de 8 puntos puede no significar lo mismo que para otro equipo, pero lo que importa es que en un mismo equipo, una US de 5 puntos, siempre sea más chica que una de 8.

Es importante tener en claro que no se estima tiempo, ya que la duración de los Sprints es fija (concepto de Timebox). Por esto, es que no se recomienda estimar el tamaño de las US en horas, para no confundir con el esfuerzo.

Tamaño vs. Esfuerzo

Como el esfuerzo (horas de trabajo) es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc. Para una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales). Por otro lado, no se debe confundir esfuerzo con calendario, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado.

Velocidad

Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.

Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración (no existe el concepto de casi-completo, es una gestión binaria, es decir, se completó o no se completó la US). Esto genera que la velocidad del equipo baje en esa iteración, pero en la siguiente, aumentará, debido a que en unos pocos días probablemente se complete la US pendiente, y se pueda continuar con otras US planificadas en la iteración.

Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones. Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.

Esta métrica permite retroalimentar el proceso de estimación, ya que un equipo maduro y con mucho trayecto, puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Reestimación

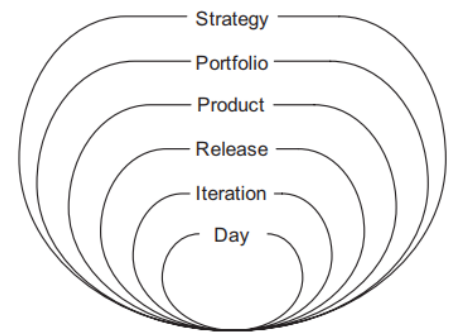
Estimar nuevamente una US es aconsejable sólo cuando la opinión acerca del tamaño relativo de una US ha cambiado, debido a que en el proceso de estimación se han detectado otras U.S. con tamaños más grandes o pequeños. (esto tiene que ver con mantener la consistencia de tamaños a la hora de estimar).

No se debería reestimar porque no se detecta un progreso como el esperado (velocidad del equipo), sino que se debe dejar que la velocidad a lo largo de sucesivas iteraciones equilibre esas inexactitudes.

Planificación

En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios, para encauzar los esfuerzos de manera correcta, y poder lograr los objetivos planteados.

“Planning is everything. Plans are nothing” → esto apunta a que lo importante de planificar, es el proceso en sí. No el hecho de realizar una documentación exhaustiva que respete determinados templates. Sí debe quedar registrado de alguna manera, idealmente bajo templates, pero sólo para servir de ayuda de memoria. Lo importante es el proceso.

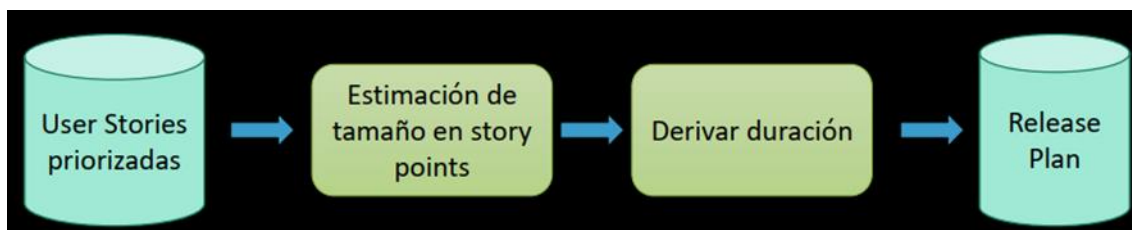


Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles:

Planificación de Release

Este proceso se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto. Este plan se va actualizando mientras el proyecto transcurre (usualmente al comienzo de cada iteración), de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.

Una vez que se priorizan las US a implementar, se estima su tamaño con Story points, y teniendo en cuenta la velocidad de trabajo del equipo, se deriva la duración de la Release o del proyecto (la cantidad de sprints necesarios para obtener el release).



Planificación de Iteración

Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada. Esto se hace teniendo en cuenta el tiempo (que es una variable constante por ser metodología ágil) y la velocidad de trabajo del equipo (el cual se mantiene constante también).

Durante esta planificación, se discute sobre las tareas que serán necesarias para transformar una US en código funcional y testeado.

Planificación del día

Este proceso permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Tamaño

Dado que el tamaño es lo que se compara, lo definimos como la **medida de cuán compleja y grande es una US**, además de cuánto trabajo es requerido para hacer o completar una Feature/US. También el tamaño es función de la incertidumbre de dicha US, definida la incertidumbre como la falta de información.

El tamaño no es lo mismo que el esfuerzo (el “quien” define esta diferencia), ya que este último indica las horas humano lineal requeridas para el desarrollo de la US. Por lo tanto, **una US de mismo tamaño puede implicar esfuerzos diferentes en función de la persona que se esté evaluando**, debido a que el esfuerzo varía dependiendo de las habilidades, conocimientos, experiencia, familiaridad con el dominio, etc.

Formas de estimar el tamaño

Existen diferentes escalas para estimar el tamaño y siempre son por comparación, entre ellas:

1. Número del 1 al 10.
2. Talles de remeras: S, M, L: suele ser utilizado como escala de estimación para los ítems del Product Backlog.
3. Serie exponencial de base 2: 2, 4, 8, 16, 32, 64, etc.
4. Serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

Una vez definida la escala, no se cambia. En caso de cambiarla, también se debe cambiar el patrón de medición.

Story Point

Es una unidad de estimación que **especifica la complejidad, incertidumbre y esfuerzo** propio del equipo respecto a una US en términos relativos **respecto a otra US**. Es lo que le da la idea del “peso” de cada US, decide cuán grande/compleja es. Por lo general esa complejidad tiende a incrementarse exponencialmente.

- **Complejidad:** que tan compleja es implementar esa US. Relacionada a la cantidad de partes y relaciones entre las mismas posee la US.
- **Esfuerzo:** es importante que estime la persona que va a hacer el trabajo, ya que el esfuerzo es distinto entre dos personas. Son las horas persona lineales que se requieren para terminar la US.
- **Incertidumbre:** es cuánta información falta para ser más adecuados en la estimación. Existe una incertidumbre técnica que es más sobre si se sabe cómo llevar a cabo lo explicitado con respecto a la tecnología a implementar y ese tipo de cosas. Y tenemos incertidumbre del negocio, si entendemos que es lo que el cliente pide y cómo llevarlo a cabo.

¿Para qué se estima?

1. En un primer momento para determinar cuántas Users se puede comprometer el equipo de trabajo a terminar en una iteración.
2. Al finalizar la iteración, contamos los puntos de historia correspondiente a las Users aceptadas por el Product Owner y obtenemos la velocidad del sprint, la cual es una de las métricas fundamentales en ambientes ágiles.

Sin embargo, es necesario aclarar que la velocidad no es una estimación sino una medición propiamente dicha. Si un equipo alcanza una velocidad sostenible en el tiempo, es decir que mantiene la velocidad en las diferentes iteraciones, permite que el equipo obtenga previsibilidad, la cual ayuda a estimar en las siguientes iteraciones.

¿Cómo se estima la duración del proyecto?

Si la estimación se realiza utilizando Story Points para medir la complejidad relativa de las User Stories, para determinar la duración de un proyecto se realiza la derivación tomando el número total de story points de sus US y dividiéndolas por la velocidad del equipo.

Poker Planning o Poker Estimation

Es una técnica de estimación en ambientes ágiles publicada por Mike Cohn. Resulta de la conjunción de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2. En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen. En otras palabras, el equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo. Un detalle es que el Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.

Escala

Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci como escala para la determinación de los puntos de historia.

1. **Valor 1 o 1/2:** son los valores más bajos y se aplican a una funcionalidad pequeña.
2. **Valor 2- 3:** funcionalidad pequeña a mediana.
3. **Valor 5:** funcionalidad media.
4. **Valor 8: funcionalidad** grande, donde nos preguntamos si es posible dividir.
1. **Valor 13 o más:** necesariamente hay que dividir ya que no ingresa en el sprint.

Procedimiento

2. Se define una lista de actividades, módulos, casos de uso, Users stories, etc.
3. Se acuerda cuál de las anteriores es la que tiene **menor grado de incertidumbre** y se la define como **canónica** asignándole 1 o 2 story point (es la más simple). Esta User es utilizada como referencia para comparar. Los equipos avanzados podrían elegir canónicas de 3 SP, con el objetivo de tener un margen para hacia arriba y hacia abajo. Por esta razón, también el ½ se incluye en la serie.
4. Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S. (comparado con la U.S. canónica y teniendo en cuenta los 3 aspectos nombrados) y se espera a que todos decidan.
5. Utilizando **juicio experto**, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones. Luego el resto del medio aportan sus opiniones. Siempre la escucha es activa, para poder aprender.
6. En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución del paso 4.

7. Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.

Consideraciones

1. Para realizar una estimación **se debe tener en cuenta el criterio de Done** y no solo la tarea de programación, es decir todo lo que hay que hacer para terminar una US y poder lidiar con el problema de las actividades omitidas.
2. El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.
3. Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Ventajas

- Permite opiniones multidisciplinarias de todo el equipo, lo cual enriquece la estimación.
- Al generar debates y discusiones, permite lograr una mejor comprensión del dominio en los miembros del equipo.
- Estudios demuestran que reunir estimaciones individuales conducen a mejores resultados, que estimar de forma grupal.
- Es una actividad que resulta divertida para el equipo.

Estimaciones en Lean

En Lean, el cual es más extremo que las metodologías ágiles, asume que las estimaciones son opcionales y en algunas ocasiones pueden convertirse en un desperdicio dado que tienen una alta probabilidad de no coincidir con la realidad, dado que cualquier variable que se modifica, modificara a la estimación también.

Estimaciones en proyectos pequeños

Para estimar proyectos pequeños lo mejor es utilizar datos históricos propios de la organización, dado que en estos casos los datos históricos de la industria no se adaptan correctamente a estas situaciones. Las estimaciones en proyectos pequeños dependen en gran medida de las capacidades de los individuos que hacen el trabajo. El SEI propone el Personal Software Process para proyectos unipersonales o 2 personas, conocidos como "mini proyectos".

Diferencias con las estimaciones en ambientes tradicionales

1. Respecto a las tres dimensiones

En Agile se tiene fijo el tiempo (lo que dura la iteración) y los recursos afectados para trabajar, dejando variable el alcance. Por lo tanto, al estimar en estos ambientes se responde a cuánto se puede comprometer el equipo, que cantidad de las características de producto se puede construir en la iteración. Se entiende, además, que en estos ambientes el producto no está completamente definido, sino que se tiene una visión de este.

En los ambientes tradicionales, el alcance es lo que está fijo y el producto se encuentra definido. A partir de esto, se deriva el resto de las variables. Por esta razón en los procesos definidos se estima primero el tamaño, ya que es la base para estimar el resto.

2. Respecto a la precisión

Las estimaciones en los ambientes tradicionales buscan ser más precisas, debido a que son las bases para luego planificar. Dado que la precisión es realmente cara, la precisión en los ambientes ágiles y en correspondencia con la filosofía Lean, es considerada un desperdicio.

3. Principal diferencia

Las estimaciones ágiles son relativas, a diferencia de las estimaciones en ambientes tradicionales que son absolutas. En Agile, las características del producto son estimadas utilizando una medida de tamaño relativa dado que las personas no somos buenas realizando estimaciones absolutas, sino que es más fácil y rápido comparar medidas. Por otra parte, la práctica de estimar mediante comparaciones permite obtener una mejor dinámica grupal y pensamiento de equipo por sobre el individuo.

4. Quien estima

En las metodologías de estimación tradicional, las estimaciones las realiza el **líder del proyecto** o en ocasiones las realiza un **experto**, por lo que su estimación no es representativa de la cantidad de trabajo que requiere una persona del equipo para terminar una funcionalidad del producto. Esta es una de las razones principales de los desvíos en los proyectos de software tradicional. En los ambientes ágiles y en concordancia con los valores y principios explicitados en el manifiesto ágil, es el **equipo** el que realiza la estimación. Lo óptimo es que cada uno estime su propio trabajo y de esta manera, es posible disminuir la probabilidad de desviaciones.

5. Confusión con la planificación

En los ambientes tradicionales existe un gran error que es asumir las estimaciones como compromisos, como si fueran planificaciones. Esto implica que las estimaciones se transforman en promesas.

Mientras que en agile, asumen que las estimaciones están inmersas en el campo de las probabilidades y por lo tanto no implican un compromiso. Una falla en la estimación no implica extender el tiempo, ya que en ágiles trabajamos con el tiempo fijo, por lo tanto, lo que se ajusta es el alcance.

6. Momentos para estimar

En el enfoque tradicional se estima al comienzo del proyecto, luego de la especificación de requerimientos y luego del diseño (debido que ahí es cuando puedo saber cuál es el tamaño y de ahí derivo todas las demás estimaciones). Realmente se estima cada vez que se modifica el alcance, ya que es la variable que se mantiene fija y de la cual se deriva el resto.

Las estimaciones a lo largo del proyecto se van haciendo cada vez más certeras. Al comienzo, pueden diferir hasta un 400%, esto se debe a:

1. Al comienzo tenemos poca información y alta incertidumbre, por lo que aumenta el riesgo de estimar erróneamente.
2. Las estimaciones son demasiado optimistas.
3. Decimos que aquel que no sabe estimar implica estimar muchas veces.

En ágiles se estima al comienzo de cada sprint.

Frameworks de SCRUM a nivel de equipo y escala

SCRUM

Scrum es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.

En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:

1. Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.